Do you have feedback for me?

Anonymous feedback form:
henrycg.com/feedback

Authenticated encryption

CS 161 Spring 2026 - Lecture 11



Textbook Chapter 8.7 & 8.8

Logistics

- Attendance codes
- Monday is a holiday! No OH.

Cryptography roadmap

	Secret-key crypto	Public-key crypto
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption ✅ CBC encryption – Authenticated encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	✅ Message auth. codes	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) ✅ Hash functions (e.g., SHA2)	– Discrete-log problem

Recap: Message authentication codes (MACs)

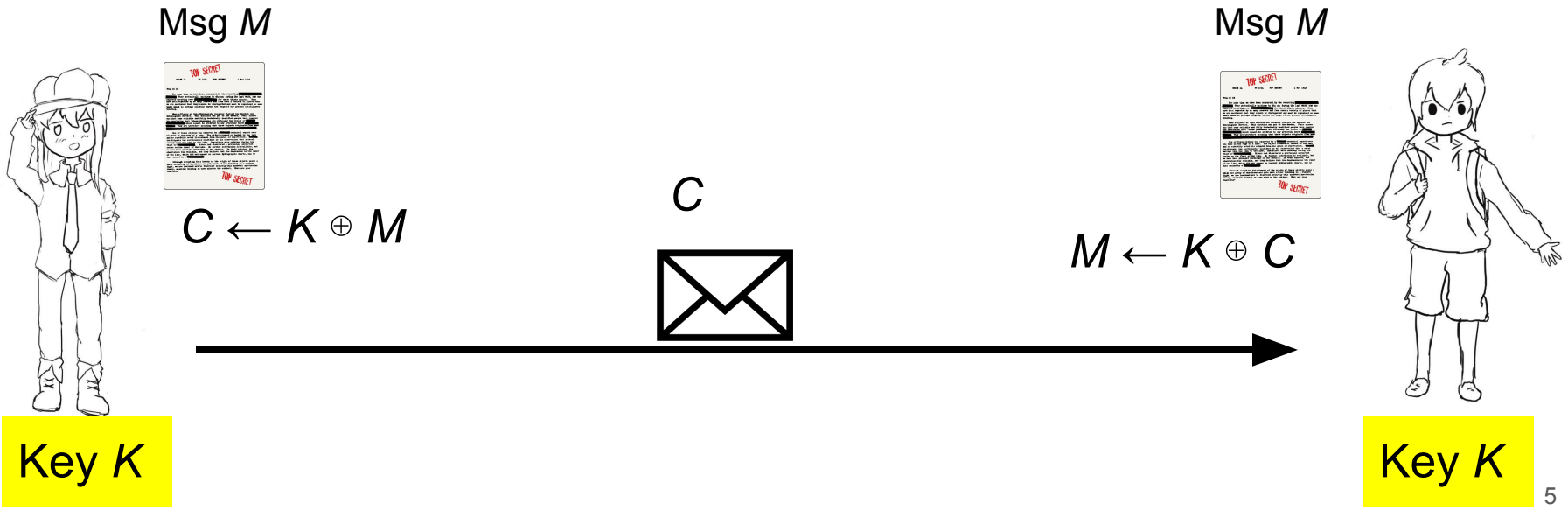
- **Syntax:** MAC(key, message) → tag
- MAC Security
  - Attacker can see tags on many messages of its choice
  - Still, attacker cannot produce a valid tag on a *single* new message
- A pseudorandom function IS a MAC for short messages
  - Can extend to MAC for long messages with collision-resistant hash
- MAC you might see: HMAC, Poly1305, …
- MACs do not provide confidentiality, do not protect against replay attacks

This time: Authenticated encryption, the “gold standard” for encryption.

Reminder:

Encryption alone provides no “integrity” guarantees

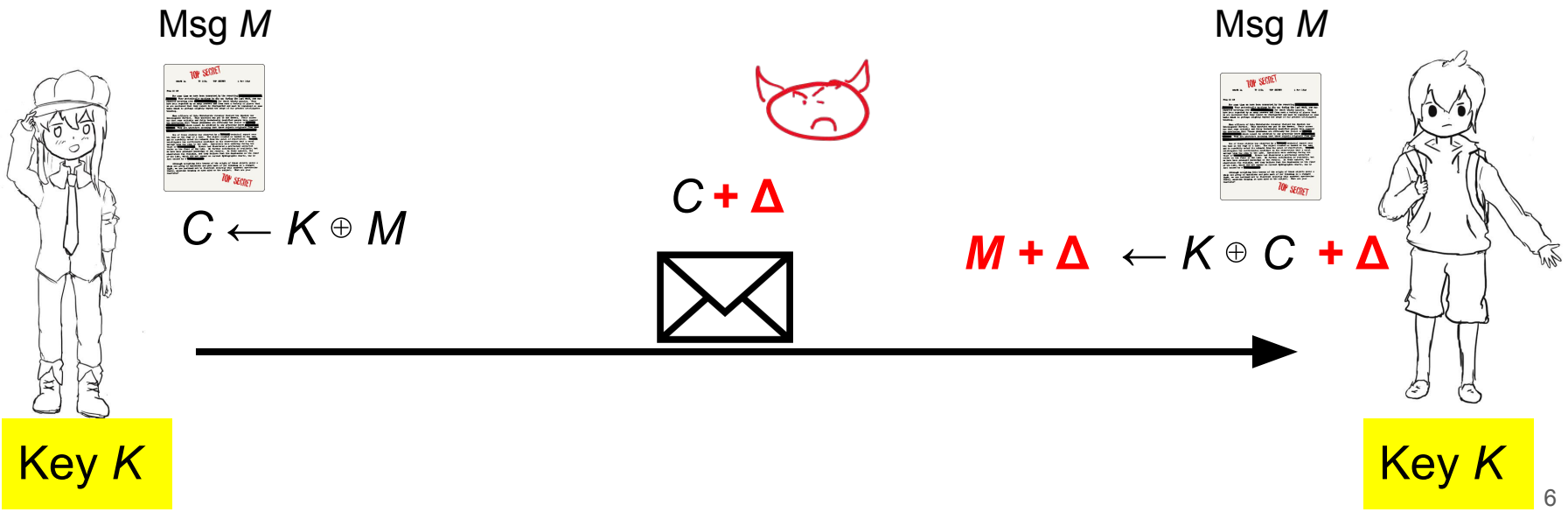
Recall one-time pad…



Reminder:

Encryption alone provides no “integrity” guarantees

Recall one-time pad…



Today: Authenticated Encryption

Authenticated encryption (AE):
A secret-key encryption scheme that simultaneously guarantees:

1. Confidentiality
2. Integrity

Even if you don't care about integrity, you still should use AE

- AE schemes provide stronger confidentiality too!
- Security against "chosen ciphertext attacks" (CCA)

AE = The gold standard for encryption = What you should be using.

So far…

- We saw encryption schemes secure against **chosen-plaintext attacks**
  - Secure even if attacker gets **ENCRYPTIONS** of messages of its choice
- What happens if attacker gets **DECRYPTIONS** of messages of its choice?
  - These are called “chosen-ciphertext attacks” (CCA)
  - CCA Security = The gold-standard notion of security for encryption schemes
  - Defining CCA security is a little subtle. These notes give one definition.
- ???

Decryptions can be powerful

- An encryption scheme may be CPA secure…
- But can reveal the entire secret key if the attacker sees the decryption of a single ciphertext of its choice

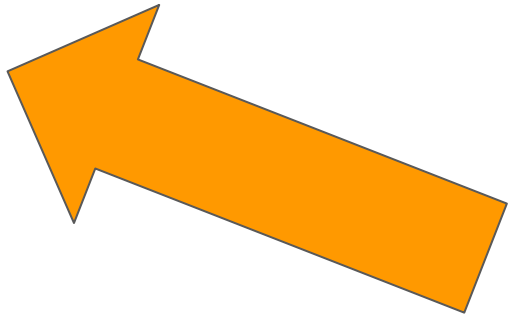
```
def Encrypt(k, m):  return CPASecure.Enc(k, m)

def Decrypt(k, c):  m = CPASecure.Dec(k, c)
                    if m == “DEBUG_VERY”: return k
                    else: return m
```

Why would we let attacker see **DECRYPTIONS** of ciphertexts of attacker's choosing?

Very common: Recipient acts on decrypted values.

```
def serve_request(packet):
    key = // load secret key
    (student_id, request) = decrypt(key, packet)
    if not student_exists(student_id):
        return "Student not found"
    else:
        handle(student_id, request)
        return "Complete."
```



Authenticated encryption (AE)

Three efficient algorithms:

- KeyGen() → K
- Enc(K, M) → C
- Dec(K, C) → M or "fail"

Must satisfy three properties

- Correctness: For all $K \leftarrow$ KeyGen(), for all M, Dec(K, Enc(K, M)) = M
- CPA Security: Security against chosen plaintext attacks
- Ciphertext integrity: Hard for an adversary to construct any "valid" ciphertext
  - Only someone with secret key can create ctexts that decrypt to non-"fail" output

Authenticated encryption: Definition

Ciphertext integrity = “Adversary can’t cook up a valid ciphertext on its own”
= Only way to get a valid ciphertext is if Alice sends it

- One-time pad does NOT have ciphertext integrity
  - Every ciphertext is valid
- AES in counter mode does NOT have ciphertext integrity
  - Every ciphertext is valid

Non-obvious fact:
AE schemes are secure against chosen-ciphertexts attacks (CCA)

- One of the reasons we like AE

Ciphertext integrity ⇒ Security against tampering



Msg M

$C \leftarrow \text{AE.Enc}(K, M)$



C^*





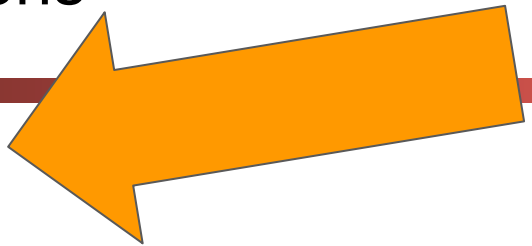
"fail" $\leftarrow \text{AE.Dec}(K, C)$



Key K

Key K

Authenticated Encryption: Constructions



Approach 1: "Black box" combination of two tools

- CPA-secure encryption // For confidentiality
- Secure message authentication code (MAC) // For integrity

  \+ Simple, reuse existing primitives

  – Could be 2x as expensive

  → *Multiple options for how to combine*

Approach 2: Clean-slate construction of AE

  \+ Can be faster

  – Requires new mode of operation

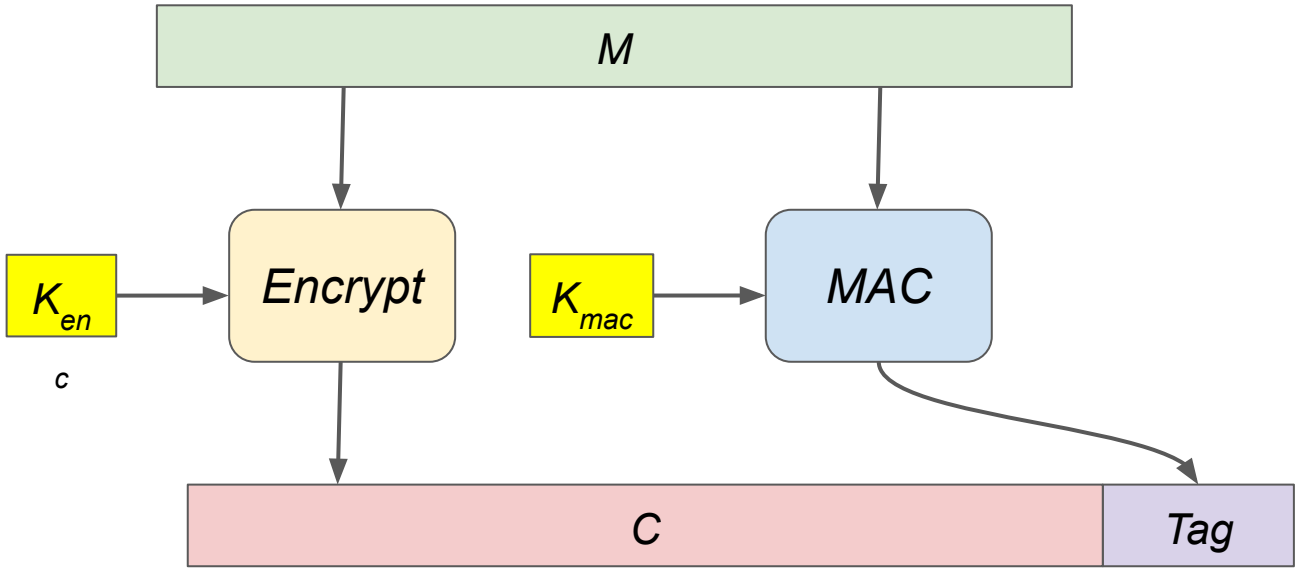
Authenticated encryption (AE) from CPA-secure encryption + MAC
Three approaches

1) Encrypt and MAC
2) Encrypt then MAC
3) MAC then encrypt

Q: Which should you use?

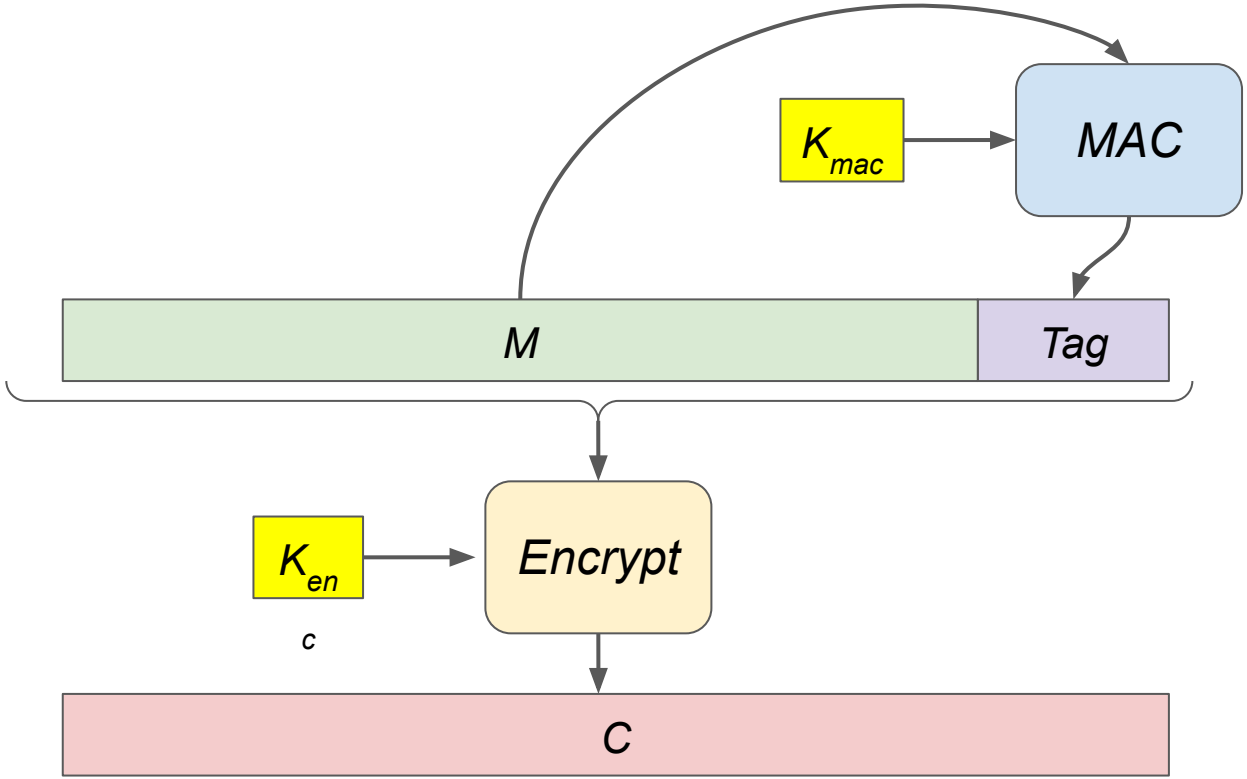
A: You should use a crypto library that makes this decision for you.

Encrypt AND MAC

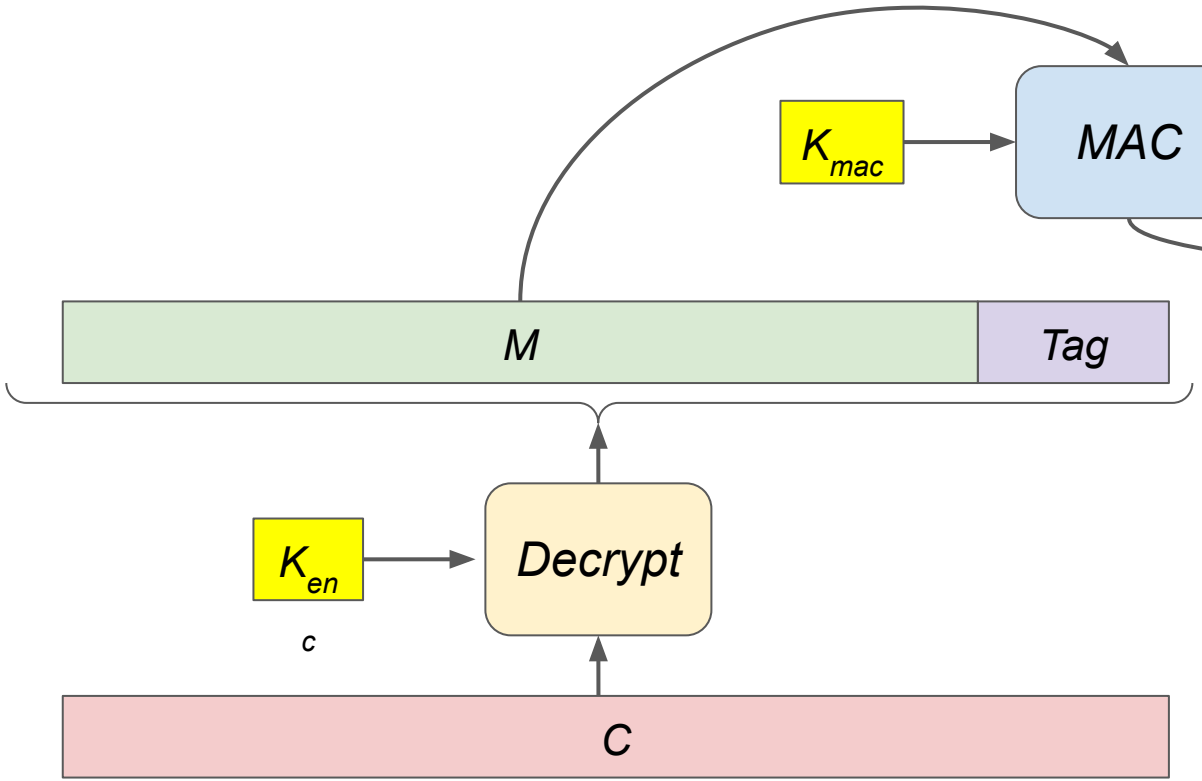


Problem: Tag may not hide message!
(MAC doesn't protect confidentiality)

MAC then Encrypt



MAC then Encrypt: Decryption



Decryption:
1) Decrypt (msg, tag)
2) Check tag on MSG

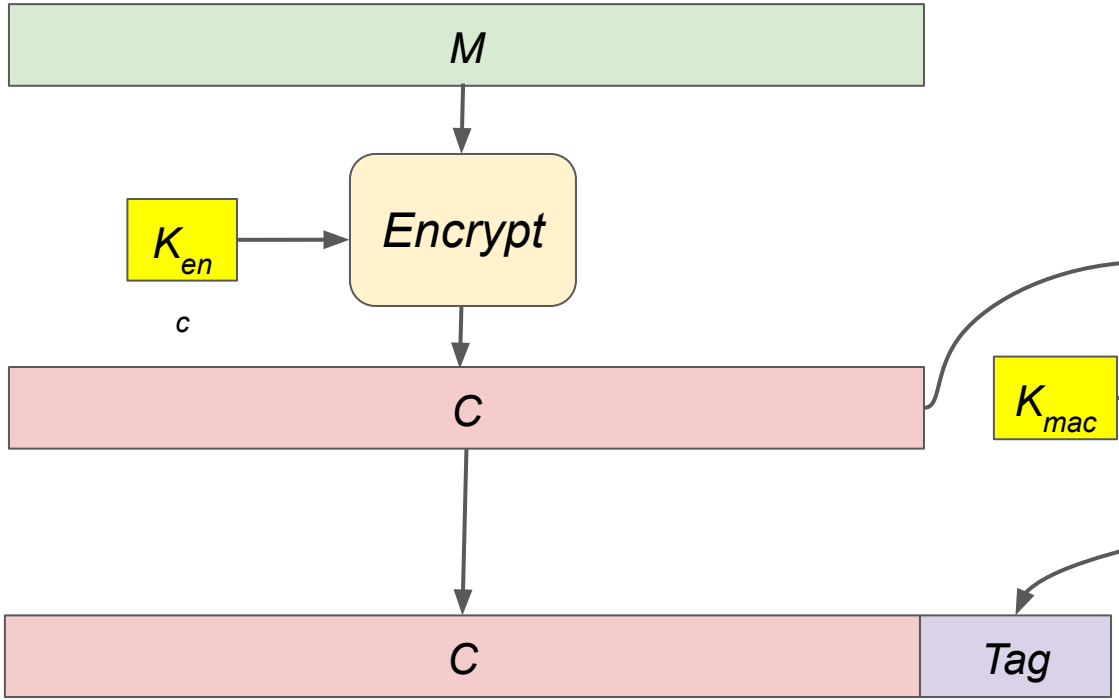
= Tag' ?

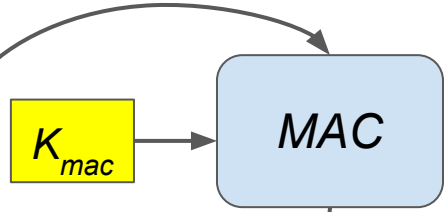
Subtle issue:
– Recipient decrypts adversarial ciphertext before checking the MAC
– Is this a problem???

TLS 1.0 “Lucky 13” Attack

- TLS: Encryption protocol that protects network traffic (HTTPS, etc.)
- TLS 1.0 uses MAC-then-encrypt: Enc(K_1, M || MAC(K_2, M))
  - The encryption algorithm is AES-CBC
- The Lucky 13 attack abuses MAC-then-encrypt to read encrypted messages
  - Guess a byte of plaintext and change the ciphertext accordingly
  - Decryption will fail, but the **time it takes to fail** is depends **on whether the guess was correct**
  - Attacker measures how server takes to complain → Learns byte of plaintext!
  - Repeat to recover whole message
- Takeaways
  - Always use AE
  - You’ll try a similar attack in Homework 2!

Encrypt then MAC 👍👍👍





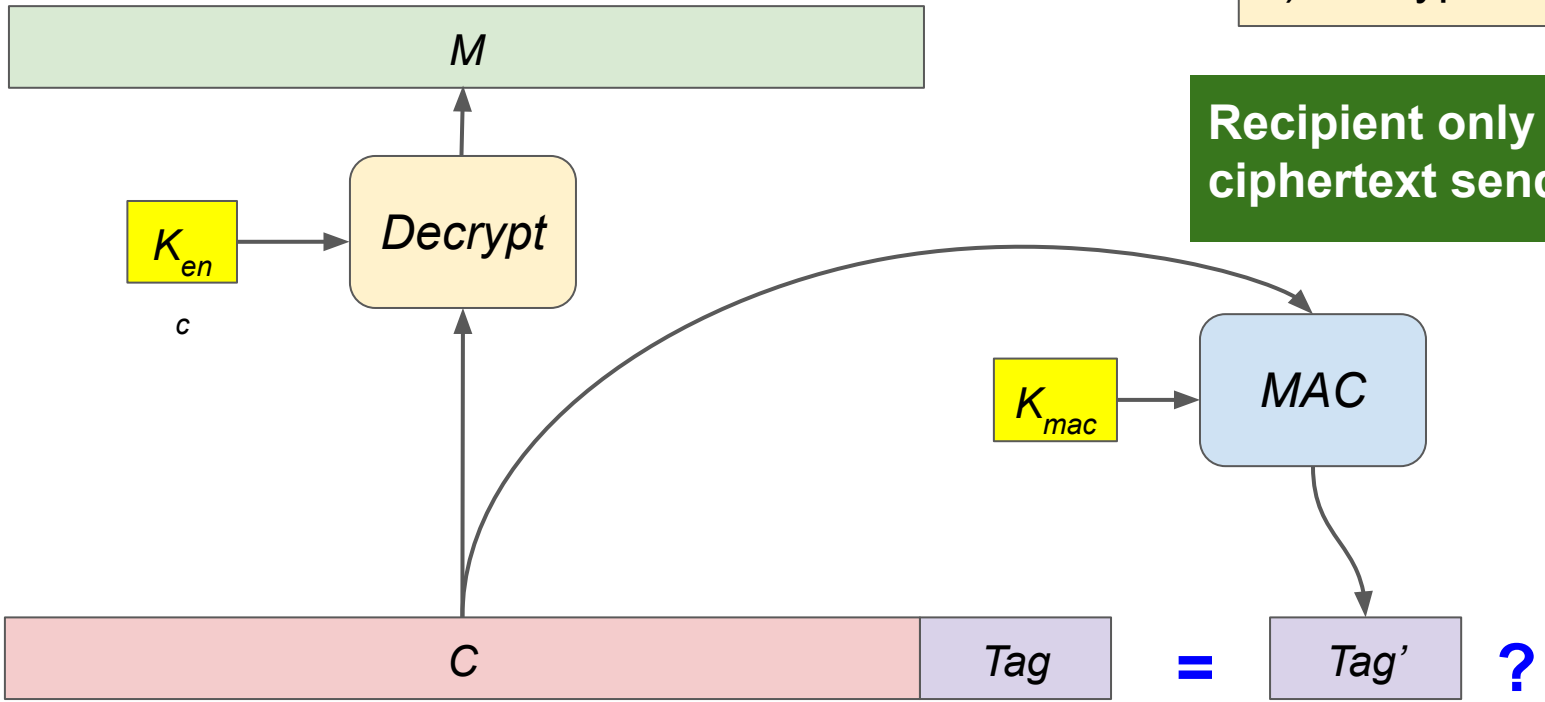
Decryption:
1) Check tag on msg
2) Decrypt ct

Recipient only decrypts ciphertext sender created.

Encrypt then MAC 👍👍👍: Decryption

Decryption:
1) Check tag on msg
2) Decrypt ct

Recipient only decrypts ciphertext sender created.

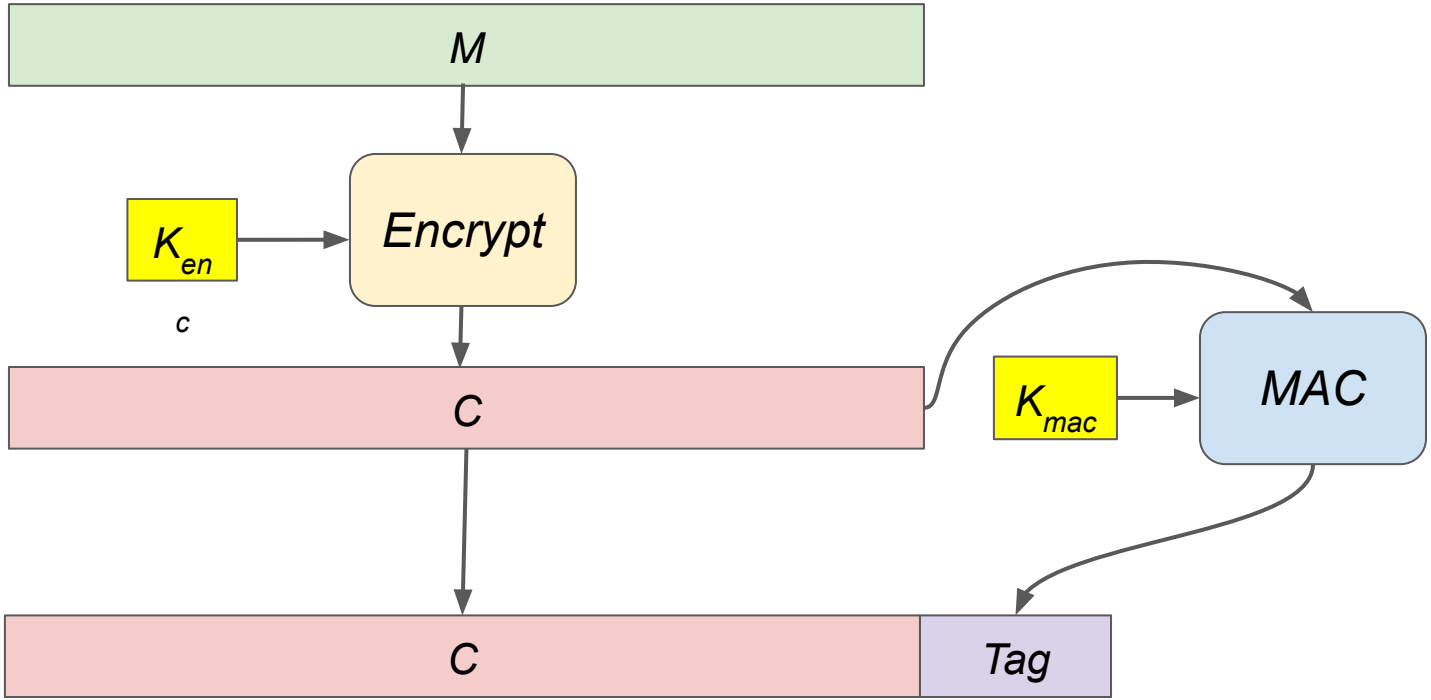


Summary

- Encrypt AND MAC
  - – Not always secure AE
  - – MAC tag can leak message
- MAC then Encrypt
  - – Not always secure AE
  - – Uses secret key to decrypt adversarial ciphertext
- Encrypt then MAC
  - + Always secure AE

    (if enc scheme is CPA Secure, MAC is secure)

Why did I use two different keys?

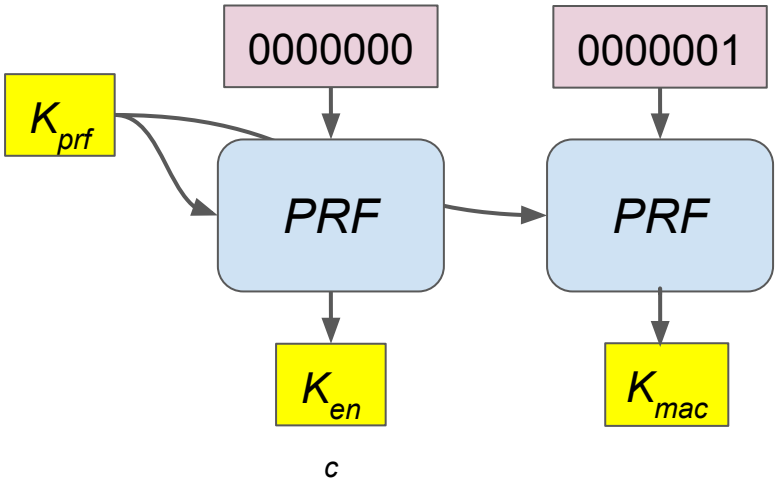


Why did I use two different keys?

- In general, encryption/MAC schemes only guarantee security when you use independent, random keys
  - Never the same key material in two different algorithms
- It's often not easy to figure out whether key reuse is safe in a piece of code
  - Often it's not.
  - Why risk it?
- Standard trick: Stretch one key into 2+ using a pseudorandom function (PRF)
  - This is always safe

e.g., HMAC

Example: Key stretching with pseudorandom function



Common in applications: HMAC, which is a MAC and a PRF

Key Reuse

Rough rule: You want a distinct key for each (*algorithm, key_owner*) pair.

- Encrypt a piece of data and MAC a piece of data?
  - Different algorithm; different key
- MAC one of Alice's messages to Bob and MAC one of Bob's messages to Alice?
  - Different key owner; different key
- Encrypt one of Alice's files and encrypt another one of Alice's files?
  - Same algorithm, same key owner; this is fine
- Encrypt user metadata, encrypt file metadata, and encrypt file data?
  - You'll have to think about this in Project 2!

Authenticated encryption with associated data (AEAD)

- AE schemes encrypt and authenticate the entire message

- AEAD schemes:
  - Encrypt part of the message
  - Authenticate the whole message
  - Examples: AES-GCM, ChaCha20-Poly1305

- Encryption routine takes two message parts as input:
  - AEAD.Enc(K, M, D) → C // D = unencrypted "associated data"
  - AEAD.Dec(K, C) → (M, D) or "fail"

- Common use: Authenticate packet headers
  - Payload of packet is encrypted

AEAD Example: Galois Counter Mode (GCM)

- **Galois Counter Mode** (**GCM**): An AEAD block cipher mode of operation
  - AES-GCM is one of the most widely used secret-key enc schemes
- Combines two tools you've seen using Encrypt-then-MAC:

  Tool 1: Randomized counter-mode encryption = CPA-secure encryption
  - Uses AES as the underlying block cipher

  Tool 2: Polynomial MAC = Secure one-time MAC
  - Uses PRF to turn a one-time MAC into many-time MAC
- Both have fast parallel implementations
- **Risk:** Reuse encryption nonce/IV ⇒ Everything breaks

Authenticated Encryption: Summary

- Chosen ciphertext attack (CCA)
  - Attacker can see ENCRYPTIONS of messages of its choosing
  - Attacker can see DECRYPTIONS of ciphertexts of its choosing

- Authenticated encryption: Guarantees confidentiality and integrity
  - Also CCA secure

- Can build AE from CPA-secure encryption and a secure MAC:
  - Encrypt-then-MAC: $\text{Enc}(K_1, M) \,||\, \text{MAC}(K_2, \text{Enc}(K_1, M))$
  - Other variations are dangerous

Next time: How can Alice and Bob agree on a secret key over an insecure channel?